MSc in Mathematical Engineering
Course: Advanced Programming for Scientific Computing

Lecturer: Luca Formaggia
Teaching Assistant: Paolo Joseph Baioni

Course project
A.Y. 2024/2025

FlowPDE: Flow-Based Generative Surrogates for Parametrized PDEs

---

Authors:

Sarper Yurtseven

Mentors:

Andrea Manzoni
Nicola Farenga
Giacomo Bottacini

Abstract

Reduced-order modelling is needed because design, optimization, control, uncertainty propagation, and inverse problems cannot afford a high-fidelity solve for every query. Most deep-learning surrogates are deterministic and trained with mean squared error, so they predict the conditional mean. This works when each input has a unique solution, but in inverse problems, partial observations, or chaotic systems, it can average incompatible solutions into an unrealistic field without showing the missing uncertainty.

This report presents `flowpde`, a PyTorch library that models the conditional distribution $p(\boldsymbol{u} \mid \boldsymbol{\mu})$ using continuous-time flows. It supports both exact maximum likelihood and flow matching with the same conditional neural ODE. Its modular design separates the dynamics, training objective, probability path, time sampler, coupling, and source distribution.

Five controlled experiments on Burgers and Darcy problems compare conditioning, network backbones, and training objectives. Removing conditioning increases the median error by 17×, although the loss curve remains similar. Surprisingly, the smallest backbone performs best on both problems. On fixed trained models, Euler sampling outperforms fourth-order Runge–Kutta by up to 8.6× at equal cost, while exact ODE integration gives 11% worse accuracy than sixteen Euler steps. Maximum-likelihood training improves test likelihood but worsens sample accuracy and costs 66× more than flow matching.

For the inverse problem, the relative L^2 error of a single sample hides the value of conditioning, while proper probabilistic scores show that it works. The learned posterior also becomes 3.9× narrower for more informative Darcy observations, capturing the problem's identifiability structure. However, the results are limited to one run per setting, include no deterministic baseline, and show no speed advantage over the reference solver at this scale.

Contents

Abstract **1**

1 Introduction **2**
- 1.1 Contributions . . . 2
- 1.2 Related work . . . 3

2 Generative AI for PDE surrogates **3**
- 2.1 Flow-based generative modeling . . . 3
- 2.2 Application to parametrized PDEs . . . 6

3 FlowPDE **8**
- 3.1 Library overview . . . 8
- 3.2 Modules . . . 9
- 3.3 Installation and testing . . . 11

4 Numerical experiments **11**
- 4.1 Setup . . . 12
- 4.2 Does the condition reach the velocity field? . . . 13
- 4.3 Backbone . . . 13
- 4.4 Accuracy against sampling cost . . . 14
- 4.5 The four components . . . 15
- 4.6 Objective: simulation-free against simulation-based . . . 16
- 4.7 Inverse problems and calibration . . . 16
- 4.8 Ill-posedness as a controlled variable . . . 19
- 4.9 The cost of a reference posterior . . . 20
- 4.10 Threats to validity . . . 20

5 Conclusions **21**

Bibliography **22**

1 Introduction

Solving a partial differential equation once is a solved problem. Solving it ten thousand times is not, and that is what the applications ask for. Design optimization sweeps a parameter space; optimal control differentiates through repeated solves; uncertainty propagation samples the input distribution; inverse problems search over inputs until the outputs match a measurement. In each case the quantity of interest is not one solution but the *map* $\boldsymbol{\mu} \mapsto u(\boldsymbol{\mu})$, queried repeatedly, and the cost of a high-fidelity discretization — the very thing that makes it trustworthy — makes that query budget unaffordable.

Reduced order modelling is the classical answer, and its central idea is an accounting one: split the work into an expensive offline stage run once and a cheap online stage run per query [26]. Deep learning has since supplied nonlinear replacements for the linear reduced subspace [9] and neural operators that learn the solution map as a mapping between function spaces [18, 19].

These methods work well on standard benchmarks. They also share a property that is easy to overlook because it is nearly universal: they are *deterministic*. Each returns one field per input, trained under a mean squared error whose population minimizer is the conditional mean $\mathbb{E}[u \mid \boldsymbol{\mu}]$. Where the solution map is a genuine function this is exactly right, since the conditional mean of a point mass is the point. The difficulty is that several of the applications motivating reduced order modelling most strongly are the ones where it is not a function. Recovering a coefficient field from sparse noisy measurements does not determine that field. Reconstructing a solution from partial observations does not determine the solution. Predicting a chaotic system far enough ahead does not determine the trajectory, even when the statistics remain well determined. In each case the conditional mean is an average over mutually incompatible candidates — and averaging solutions of a nonlinear equation does not give a solution, it gives a smooth field that satisfies nothing. The model returns it with no indication that anything was lost.

This report takes the alternative route: model the conditional distribution $p(u \mid \boldsymbol{\mu})$ and draw samples from it. Individual samples can then be sharp and equation-consistent rather than averaged, and the spread across samples estimates what the data left undetermined. The vehicle is the family of continuous-time flow-based generative models — continuous normalizing flows [3, 11] and, above all, the simulation-free flow matching objectives that have largely displaced them [1, 20, 23]. Two features suit surrogate modelling specifically. Sampling is the solution of an ordinary differential equation, so the number of function evaluations is an accuracy-against-cost dial that can be turned after training, on fixed weights. And conditional training is *amortized*: one training run yields a sampler for the whole family $\{p(\cdot \mid \boldsymbol{\mu})\}_{\boldsymbol{\mu}}$, which is the offline/online decomposition of reduced order modelling recovered from the machine-learning side.

The contribution is a library, `flowpde`, rather than a single algorithm. That choice reflects where the difficulty currently lies. The design space of flow-based PDE surrogates is large — interpolation path, time schedule, noise–data coupling, source distribution, backbone, training objective, solver and step count — and published comparisons between its axes are sparse, partly because every variant tends to arrive as its own codebase. A library in which these are orthogonal, swappable components makes the comparison a matter of configuration rather than of reimplementation.

1.1 Contributions

- **The flow is separated from the objective.** The dynamics are one object and the loss fitting them is another, so the same conditional neural ODE can be trained by exact maximum likelihood or by flow matching with no change to model, data or sampler. Comparing simulation-based against simulation-free training becomes a controlled experiment instead of a comparison between two codebases (Section 3.1).
- **Flow matching is factored into four orthogonal components.** Path, time sampler, coupling and source are independently substitutable, so the variants of Section 2.1.2 are *configurations* of one implementation rather than subclasses of it (Section 3.2), which Section 4.5 exploits to compare six of them under one controlled protocol.
- **Reflow's correctness invariant is enforced structurally.** As Remark 4 explains, reflow straightens trajectories only if training consumes the same source point that generated each target; resampling noise independently makes it a no-op that is invisible in the loss curve. Making the source an explicit component turns that invariant into something a regression test can pin.
- **Forward and inverse generators with controllable ill-posedness.** Poisson, Burgers and Darcy problems are generated through one spectral pipeline, in either direction, with observation noise and random masking as explicit knobs — swept in Section 4.8 from a posterior that is no better than the prior to one that is genuinely informative.

- **Distributional evaluation rather than relative L^2 alone.** Proper scoring rules, coverage, rank histograms, spread–skill ratios and a variance decomposition, in physical units (Section 2.2.2).
- **A controlled study across the design space.** Conditioning mechanism, backbone, training objective, flow-matching components and observation quality are varied on shared data, normalization, optimizer and sampler, so each comparison isolates one axis; the cheap cells are replicated at matched seeds (Section 4).

1.2 Related work

The bibliography is deliberately short: an entry appears only where a specific claim depends on it, and roughly half correspond to something implemented in the library itself. Neighbouring work that could be listed for coverage has been left out rather than cited unread.

Flow-based generative modelling. Normalizing flows [27] became continuous with neural ODEs [3] and scalable with free-form dynamics [11]; diffusion and score-based models [14, 29] then dominated for several years; and flow matching [20], rectified flow [23] and stochastic interpolants [1] unified and simplified the lot, with minibatch-OT couplings [32] and time-schedule design [7] as the principal refinements. Section 2.1 develops this material; Lipman et al. [21] is the current reference treatment.

Generative models for PDEs. PDE-Refiner [22] recasts a solver update as a short denoising process to recover low-amplitude spectral content that a mean squared error ignores; Kohl et al. [17] benchmark autoregressive conditional diffusion for turbulence and find its long-rollout statistics markedly better than deterministic predictors'; and GenCast [25] demonstrates the approach at operational scale for weather. Two features of this literature bear on what follows. It is predominantly *diffusion* based rather than flow matching based, and predominantly concerned with *temporal* problems, where the probabilistic argument is easiest to make. Parametrized elliptic and parabolic problems treated in both directions, with the flow matching objective and with calibration reported rather than assumed, are less covered, and that is the gap this work sits in. The inverse half has one direct antecedent: Wildberger et al. [33] train a conditional flow matching model as an amortized posterior estimator, which is the construction Section 2.2.2 adopts.

Section 2 reviews flow-based generative modelling and its application to parametrized PDEs. Section 3 describes the library. Section 4 reports the numerical experiments.

2 Generative AI for PDE surrogates

This section sets up the machinery the report rests on. Section 2.1 defines a flow-based generative model and gives three ways to train it. Section 2.2 connects that to partial differential equations: what the distribution is over, and when modelling one is worth the trouble.

2.1 Flow-based generative modeling

An unknown distribution q on $\mathbb{R}^d$ is known only through samples, and we want more samples from it. The flow-based answer is to *transport* a simple reference $\rho_0 = \mathcal{N}(\mathbf{0}, I)$ onto q.

Let $\boldsymbol{v}_t(\boldsymbol{x})$ be a time-dependent velocity field on $[0,1] \times \mathbb{R}^d$ and let ψ_t be its flow map, defined by

$$\frac{\mathrm{d}}{\mathrm{d}t}\psi_t(\boldsymbol{x}_0) = \boldsymbol{v}_t\big(\psi_t(\boldsymbol{x}_0)\big), \qquad \psi_0(\boldsymbol{x}_0) = \boldsymbol{x}_0. \tag{1}$$

If $\boldsymbol{v}$ is Lipschitz in $\boldsymbol{x}$, Picard–Lindelöf makes ψ_t a diffeomorphism for every t, with the inverse obtained by integrating backwards. *Invertibility is therefore automatic*, which Section 2.1.1 shows to be the decisive advantage of the continuous formulation.

Pushing ρ_0 through the flow gives a curve of measures $p_t = (\psi_t)_{\#}\rho_0$, the *probability path*, equivalently the solution of the continuity equation

$$\partial_t p_t(\boldsymbol{x}) + \nabla\cdot\big(p_t(\boldsymbol{x})\,\boldsymbol{v}_t(\boldsymbol{x})\big) = 0, \qquad p_0 = \rho_0. \tag{2}$$

We say $\boldsymbol{v}$ *generates* p_t. The generative task is the boundary condition $p_1 = q$. Many fields satisfy it, so the methods below differ not in what they seek but in the loss used to find it.

Remark 1 (One object, three training routes)**.** *The three subsections that follow all describe a velocity field $\boldsymbol{v}_{\boldsymbol{\theta}}$ and the path it generates. They differ only in the objective. Section 2.1.1 maximizes exact likelihood, which needs an ODE solve inside every gradient step. Section 2.1.2 regresses against a closed-form target*

and never solves an ODE while training. Section 2.1.3 keeps that regression but changes which pairs of points are joined, so as to straighten the trajectories. Continuous normalizing flows, flow matching and rectified flow are one model family and three training procedures — in FlowPDE, literally the same class fitted by different objectives.

2.1.1 Continuous normalizing flows

A classical normalizing flow [27] composes simple invertible layers, so the change-of-variables formula gives the exact density

$$\log p_1(\boldsymbol{x}) = \log \rho_0\big(\psi^{-1}(\boldsymbol{x})\big) - \sum_{k=1}^{K} \log \left| \det \frac{\partial \psi^{(k)}}{\partial \boldsymbol{x}} \right|. \tag{3}$$

That exactness is what distinguishes normalizing flows from variational autoencoders and adversarial networks. The price is the determinant: a general one costs $\mathcal{O}(d^3)$, so layers must be triangular or coupling-based, and the architecture ends up designed around the Jacobian rather than around the problem.

Chen et al. [3] removed the trade-off by taking depth to infinity. Parametrize $\boldsymbol{v}_{\boldsymbol{\theta}}$ in (1) by any network at all; invertibility now comes from uniqueness of ODE solutions. The log-determinant is replaced by the *instantaneous change of variables*, $\mathrm{d}\log p_t / \mathrm{d}t = -\operatorname{tr}(\partial \boldsymbol{v}_t / \partial \boldsymbol{x})$ along a trajectory, which follows from (2) by expanding the divergence and dividing by p_t. An $\mathcal{O}(d^3)$ determinant has become an $\mathcal{O}(d)$ trace and a sum over layers has become an integral:

$$\log p_1(\boldsymbol{x}_1) = \log \rho_0(\boldsymbol{x}_0) - \int_0^1 \operatorname{tr}\left(\frac{\partial \boldsymbol{v}_{\boldsymbol{\theta}}}{\partial \boldsymbol{x}}\big(\boldsymbol{x}(t), t\big) \right) \mathrm{d}t. \tag{4}$$

Both terms come from one backward solve of the augmented system $[\boldsymbol{x}, \log p]$, and training minimizes

$$\mathscr{L}_{\mathrm{NLL}}(\boldsymbol{\theta}) = -\,\mathbb{E}_{\boldsymbol{x}_1 \sim q}\big[\log p_1^{\boldsymbol{\theta}}(\boldsymbol{x}_1)\big]. \tag{5}$$

The trace still costs d Jacobian-vector products. FFJORD [11] removes that with the Hutchinson estimator [15]: for $\mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{0}$, $\operatorname{Cov}(\boldsymbol{\varepsilon}) = I$,

$$\operatorname{tr}(A) = \mathbb{E}_{\boldsymbol{\varepsilon}}\big[\boldsymbol{\varepsilon}^{\top} A\, \boldsymbol{\varepsilon}\big], \qquad \operatorname{tr}\left(\frac{\partial \boldsymbol{v}_{\boldsymbol{\theta}}}{\partial \boldsymbol{x}}\right) \approx \frac{1}{M} \sum_{m=1}^{M} \boldsymbol{\varepsilon}_m^{\top} \left(\frac{\partial \boldsymbol{v}_{\boldsymbol{\theta}}}{\partial \boldsymbol{x}} \boldsymbol{\varepsilon}_m \right), \tag{6}$$

each term one backward pass. It is unbiased for any $M \geq 1$, but its variance grows with d, which is why M is sometimes raised.

Three costs follow, and together they motivate Section 2.1.2. Training *requires simulation*: every gradient step contains tens to hundreds of network evaluations. Nothing in (5) rewards well-behaved dynamics, since the likelihood depends on $\boldsymbol{v}_{\boldsymbol{\theta}}$ only through the map at $t = 1$, so the field stiffens during training and the cost per step *grows* [11]. And the usual remedies — kinetic-energy or Jacobian penalties — are instructions to prefer straight-line transport, anticipating as an afterthought what flow matching gets by construction. Conditioning changes nothing structurally: write $\boldsymbol{v}_{\boldsymbol{\theta}}(\boldsymbol{x}, t; \boldsymbol{c})$ and replace q by $q(\cdot \mid \boldsymbol{c})$.

2.1.2 Flow matching

If we already knew a path p_t from ρ_0 to q and a field $\boldsymbol{u}_t$ generating it, fitting $\boldsymbol{v}_{\boldsymbol{\theta}}$ would be plain regression with no ODE solve anywhere [20]. We know neither. The resolution is to build the path as a mixture of elementary paths that are known in closed form. Condition on $\boldsymbol{x}_1 \sim q$ and pick a conditional path with

$$p_0(\cdot \mid \boldsymbol{x}_1) = \rho_0, \qquad p_1(\cdot \mid \boldsymbol{x}_1) \approx \delta_{\boldsymbol{x}_1}, \tag{7}$$

so each carries the whole reference onto one data point. Marginalizing gives the boundary conditions we want,

$$p_t(\boldsymbol{x}) = \int p_t(\boldsymbol{x} \mid \boldsymbol{x}_1)\, q(\boldsymbol{x}_1)\, \mathrm{d}\boldsymbol{x}_1, \qquad p_0 = \rho_0, \quad p_1 = q. \tag{8}$$

The conditional field $\boldsymbol{u}_t(\boldsymbol{x} \mid \boldsymbol{x}_1)$ is analytic; the marginal one,

$$\boldsymbol{u}_t(\boldsymbol{x}) = \int \boldsymbol{u}_t(\boldsymbol{x} \mid \boldsymbol{x}_1) \frac{p_t(\boldsymbol{x} \mid \boldsymbol{x}_1)\, q(\boldsymbol{x}_1)}{p_t(\boldsymbol{x})}\, \mathrm{d}\boldsymbol{x}_1, \tag{9}$$

is an intractable integral against the posterior over which data point produced $\boldsymbol{x}$. It never has to be evaluated.

Theorem 1 (Conditional flow matching; 20)**.** *Let p_t and $\boldsymbol{u}_t$ be the marginals* (8) *and* (9)*. Then $\boldsymbol{u}_t$ generates p_t, and with*

$$\mathscr{L}_{\mathrm{CFM}}(\boldsymbol{\theta}) = \mathbb{E}_{t,\, \boldsymbol{x}_1 \sim q,\, \boldsymbol{x} \sim p_t(\cdot \mid \boldsymbol{x}_1)} \left\| \boldsymbol{v}_{\boldsymbol{\theta}}(\boldsymbol{x}, t) - \boldsymbol{u}_t(\boldsymbol{x} \mid \boldsymbol{x}_1) \right\|^2, \tag{10}$$

the gradients of the marginal and conditional objectives coincide, so the two have the same minimizers in $\boldsymbol{\theta}$.

The proof expands both squared norms: the $\|\boldsymbol{v}_{\boldsymbol{\theta}}\|^2$ terms agree because $\boldsymbol{x}$ has the same marginal law under both expectations, the $\boldsymbol{\theta}$-free terms vanish under the gradient, and the cross terms are equal by substituting (9). The consequence is that a target we cannot compute is replaced by one we can, at no cost in the minimizer. The model still cannot fit the conditional target exactly — many pairs give the same $\boldsymbol{x}_t$ — so it converges to their conditional average, which is $\boldsymbol{u}_t$. *The residual therefore never reaches zero*, and that irreducible floor is why Section 3.2 selects models on sampled error instead of on the loss.

Gaussian paths, and diffusion as a special case. The standard conditional family is Gaussian, $p_t(\boldsymbol{x} \mid \boldsymbol{x}_1) = \mathscr{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_t(\boldsymbol{x}_1), \sigma_t(\boldsymbol{x}_1)^2 I)$, whose generating field is $\boldsymbol{u}_t(\boldsymbol{x} \mid \boldsymbol{x}_1) = (\sigma_t'/\sigma_t)(\boldsymbol{x} - \boldsymbol{\mu}_t) + \boldsymbol{\mu}_t'$. Different schedules $(\boldsymbol{\mu}_t, \sigma_t)$ recover different known models: the variance-preserving and variance-exploding schedules of score-based diffusion [14, 29] are two, which places diffusion inside this framework — its probability-flow ODE is (1) for a particular Gaussian path. Those schedules are not the natural ones, though; they come from a noising process that must run to $t \to \infty$ to forget the data, and are consequently curved.

Lipman et al. [20] advocate the optimal-transport path instead,

$$\boldsymbol{\mu}_t(\boldsymbol{x}_1) = t\, \boldsymbol{x}_1, \qquad \sigma_t = 1 - (1 - \sigma_{\min})t, \tag{11}$$

where mean and standard deviation move at constant rate. As $\sigma_{\min} \to 0$ this collapses to

$$\boldsymbol{x}_t = (1 - t)\, \boldsymbol{x}_0 + t\, \boldsymbol{x}_1, \qquad \boldsymbol{u}_t(\boldsymbol{x}_t \mid \boldsymbol{x}_0, \boldsymbol{x}_1) = \boldsymbol{x}_1 - \boldsymbol{x}_0 : \tag{12}$$

the straight segment from a noise sample to a data sample, and its constant velocity. Nothing in the final objective is more complicated than that.

Remark 2 (Interpolation and velocity must agree)**.** *In* (12) *the target velocity is the* exact time derivative *of the interpolant. This is a requirement, not a convention: if the two disagree, the model sees points from one path and regresses the velocity of another, and converges to something generating neither. Any implementation offering several paths must treat $\boldsymbol{u}_t = \partial_t \boldsymbol{x}_t$ as a tested invariant, which is how FlowPDE handles it.*

Couplings. Theorem 1 assumes the independent coupling $\pi = \rho_0 \otimes q$, but it holds for any π with the right marginals [1, 32], and the choice matters. Under independence the target at a given $(\boldsymbol{x}_t, t)$ varies enormously across draws, and that variance sets the noise floor. Minibatch optimal transport [32] reduces it by pairing within a batch: solve

$$\sigma^\star = \arg\min_{\sigma \in S_B} \sum_{i=1}^{B} \left\| \boldsymbol{x}_0^{(\sigma(i))} - \boldsymbol{x}_1^{(i)} \right\|^2 \tag{13}$$

and train on the reordered pairs, so segments shorten and targets agree more closely. Two caveats: the minibatch plan is a *biased* estimator of the population plan, with bias falling in B, and (13) costs $\mathscr{O}(B^3)$ exactly.

The stochastic-interpolant view [1], which posits the interpolant rather than the density, makes two further axes explicit: the *source* $\boldsymbol{x}_0$ is a first-class random variable rather than the limit of a noising process, and the *coupling* is free. Both are components of the library in Section 3.

Time sampling and conditioning. The derivation gives $t \sim \mathscr{U}[0, 1]$, but the regression is not equally hard at every t; reweighting concentrates capacity where the residual is large, the logit-normal schedule of Esser et al. [7] being the common choice. Conditioning again changes nothing structurally: append $\boldsymbol{c}$ and hold it fixed while $\boldsymbol{x}$ evolves, since (1) moves the state and not the condition. Sampling is one draw and one solve of (1), which is where the step count becomes an accuracy/cost dial.

2.1.3 Rectified flows

Sampling cost is the number of function evaluations, and that is governed by how curved the trajectories are. A straight path traversed at constant speed is integrated exactly by one Euler step.

Definition 1 (Straightness; 23)**.** *For a flow with trajectories $\boldsymbol{z}_t$,*

$$S = \int_0^1 \mathbb{E}\big[\|(\boldsymbol{z}_1 - \boldsymbol{z}_0) - \boldsymbol{v}_{\boldsymbol{\theta}}(\boldsymbol{z}_t, t)\|^2\big]\,\mathrm{d}t, \tag{14}$$

so $S = 0$ exactly when every trajectory is a straight line at constant speed.

Note what this measures: deviation from the *chord*, not the spread of speeds. A field turning at constant speed is curved and must not score as straight.

The conditional paths of (12) are already straight, but they *cross*: two segments can pass through the same $(\boldsymbol{x}_t, t)$ with different velocities. The marginal field (9) takes one value there, the posterior average, and ODE trajectories cannot intersect at all. The flow therefore realises a different, non-crossing pairing along curved paths, so $S > 0$. Straight conditional paths do not give a straight marginal flow.

The reflow procedure. Liu et al. [23] replace the independent coupling with one the model itself induces. Given a trained $\boldsymbol{v}_{\boldsymbol{\theta}}$, draw $\boldsymbol{z} \sim \rho_0$, integrate (1) accurately to get $\boldsymbol{x}_1' = \mathrm{ODE}(\boldsymbol{z})$, and retrain on the pairs $(\boldsymbol{z}, \boldsymbol{x}_1')$. That coupling is deterministic and, being induced by an ODE flow, non-crossing — so the same objective can now be fitted by straight lines. Repeating gives the k-rectified flow. Marginals are preserved at every iteration, and the transport cost does not increase.

Remark 3 (Rectified flow is not the same thing as reflow)**.** *The two are routinely conflated. Training with a linear path and independent coupling — "1-rectified flow" — is mathematically identical to standard flow matching and straightens nothing. Reflow is the retraining procedure, and it is what produces the straightening. The distinction is structural in FlowPDE (Section 3.2).*

Remark 4 (The pairing is the method)**.** *Reflow works only if training consumes the* same $\boldsymbol{z}$ *that generated each target. The pairing is the entire content of the procedure. Resampling noise independently decouples the pairs and reduces reflow to training on the model's own samples, which straightens nothing — and the loss curve looks identical either way, so the bug is invisible in training diagnostics. This is why the source of $\boldsymbol{x}_0$ is an explicit component in Section 3 rather than an implicit $\mathcal{N}(\mathbf{0}, I)$ inside the loss.*

Reflow is not distillation: it retrains the same objective on a new coupling and keeps a valid ODE at every stage, whereas distillation fits a student to imitate a teacher's endpoints and abandons the ODE structure.

2.2 Application to parametrized PDEs

Nothing above mentions a differential equation. This subsection supplies the missing half: what the distribution is over, and why one would want a distribution at all.

The parametrized problem. Let $\boldsymbol{\mu} \in \mathscr{P}$ be a parameter — physical coefficients, or a field such as a diffusion coefficient, a forcing term or an initial condition — and consider

$$\mathscr{A}\big(u; \boldsymbol{\mu}\big) = 0 \ \text{ in } \Omega, \qquad \mathscr{B}(u) = 0 \ \text{ on } \partial\Omega, \tag{15}$$

assumed well posed, so the solution map $\mathscr{S} : \boldsymbol{\mu} \mapsto u(\boldsymbol{\mu})$ is single-valued. After discretization its image is the *solution manifold*: a set parametrized by $\mathscr{P}$, of intrinsic dimension at most $\dim \mathscr{P}$, sitting in a space of dimension N_h that may run to millions. Reduced order modelling exists because that gap is enormous and because the applications — optimization, control, inverse problems, uncertainty propagation — are *many-query* [26].

The classical construction approximates the manifold by a linear subspace and closes the reduced problem by Galerkin projection. Its limitation is structural: linear approximation succeeds only if the Kolmogorov n-width decays quickly, and for transport-dominated problems it does not, so n grows until the reduced model is no longer reduced. Deep-learning reduced order models replace the subspace with a learned nonlinear decoder [9]; neural operators learn $\mathscr{S}$ as a map between function spaces [18, 19].

Why a distribution. As Section 1 argued, these methods are deterministic and under a mean squared error return the conditional mean $\mathbb{E}[\boldsymbol{u} \mid \boldsymbol{\mu}]$ — which is exactly right when $\mathscr{S}$ is a function and an average over incompatible solutions when it is not. The generative response is to model $p(\boldsymbol{u} \mid \boldsymbol{\mu})$ and sample. The mapping onto Section 2.1 needs no new machinery: the conditioning variable is $\boldsymbol{c} = \boldsymbol{\mu}$, the target is $\boldsymbol{x}_1 = \boldsymbol{u}_h$, and the trained field $\boldsymbol{v}_{\boldsymbol{\theta}}(\boldsymbol{x}, t; \boldsymbol{\mu})$ is an amortized sampler for the whole family $\{p(\cdot \mid \boldsymbol{\mu})\}$. That amortization is the offline/online split reached from the machine-learning side: one training run buys posterior samples for any new $\boldsymbol{\mu}$ at one ODE solve.

2.2.1 Forward problems

The forward problem is (15) read left to right. The benchmarks used later are of this type — the initial-to-final map for Burgers and the coefficient-and-source map $(\kappa, f) \mapsto u$ for Darcy, with Poisson appearing as a dataset rather than a trained model — and each is exactly the operator-learning task on which deterministic methods are strongest. The comparison should be made in that awareness: the Fourier neural operator reaches around one per cent relative L^2 on the standard Darcy and Burgers benchmarks [19], and a generative surrogate that merely matches it has demonstrated nothing.

Generative models are nonetheless applied to forward problems, for reasons that are not about uncertainty. PDE-Refiner [22] observes that a mean squared error is dominated by the high-amplitude low-frequency part of the solution and effectively ignores the low-amplitude high-frequency part — which is what governs long rollouts — and recasts the update as a few denoising steps to force attention across the spectrum. Kohl et al. [17] find autoregressive conditional diffusion keeps turbulent statistics faithful long past the point where deterministic predictors have collapsed to over-smoothed fields. GenCast [25] does this at operational scale for weather. The common thread: where the pointwise map is unstable but its statistics are not, a distributional target is better posed.

Remark 5 (The forward posterior is a Dirac)**.** *For a well-posed forward problem $p(\boldsymbol{u} \mid \boldsymbol{\mu})$ is a point mass at $\mathscr{S}(\boldsymbol{\mu})$. Any spread a conditional sampler exhibits is therefore* model error*, not physical uncertainty, and reporting it as uncertainty quantification is a category error rather than an imprecision. The uncertainty claims of this report rest on Section 2.2.2; the forward experiments are reported as verification that the operator is learned at all.*

Given Remark 5, what the forward setting contributes is measurement. It has ground truth for every input, so the accuracy of the learned transport can be checked directly; and since sampling cost is the number of function evaluations, it is where the accuracy-against-cost curve can be traced on fixed weights. Straightness, reflow and solver choice are properly evaluated here. The probabilistic claims are not.

2.2.2 Inverse problems

The inverse problem reverses (15): recover the data from measurements of the solution. Write the observation as

$$\boldsymbol{y} = \mathscr{O}\big(\mathscr{S}(\boldsymbol{\mu})\big) + \boldsymbol{\eta}, \qquad \boldsymbol{\eta} \sim \mathscr{N}(\mathbf{0}, \sigma^2 I), \tag{16}$$

with $\mathscr{O}$ typically a restriction to part of the domain. The Bayesian formulation [30] seeks

$$p(\boldsymbol{\mu} \mid \boldsymbol{y}) \;\propto\; p(\boldsymbol{y} \mid \boldsymbol{\mu})\, p(\boldsymbol{\mu}), \tag{17}$$

which is the right object precisely because the problem is ill-posed. Three mechanisms make it so: $\mathscr{O}$ discards information by construction, the noise destroys more, and $\mathscr{S}$ need not be injective. The result is not a near-degenerate posterior that a point estimate approximates adequately — it is a posterior with real width, and reporting its mean discards the answer.

Darcy flow makes this concrete. The coefficient κ enters $-\nabla \cdot (\kappa \nabla u) = f$ only through the flux, so wherever ∇u is small the data barely constrain it, and many coefficient fields explain the same observation. Identifiability consequently *varies over the domain.* A posterior sample set represents that structure; a conditional mean flattens it into one smooth field whose smoothness is an artefact of averaging.

The classical route to (17) is Markov chain Monte Carlo, whose every likelihood evaluation needs a forward solve and which must be rerun for each new observation. There are two ways to remove that cost. *Route (a)* trains an unconditional model of $p(\boldsymbol{u})$ and steers its sampler towards agreement with $\boldsymbol{y}$ at inference time [4]; one prior then serves any observation operator, but each new observation costs a full guided sampling run with likelihood gradients, and the guidance term is an approximation whose error is hard to control. *Route (b)* trains a conditional model directly on pairs $(\boldsymbol{y}, \boldsymbol{\mu})$ from the simulator, learning $p(\boldsymbol{\mu} \mid \boldsymbol{y})$ outright — simulation-based inference, needing samples from the forward model but never its likelihood. Wildberger et al. [33] show that the objective of Theorem 1 gives posterior estimators scaling better than the normalizing-flow architectures previously used. Its cost is that the model is tied to the observation operator and noise level it was trained on.

FlowPDE takes route (b), because amortized inference *is* the offline/online decomposition of reduced order modelling: one expensive offline stage buys an online stage in which a new observation yields posterior samples through a single ODE solve, with no likelihood evaluation and no gradient through the physics. Route (a) has no online stage in that sense. This is a choice with a cost, not an obvious default.

How one would know the posterior is right. A model that returns a distribution must be scored as one — a point regularly skipped in this literature, where relative L^2 of the ensemble mean is often the only number reported. That quantity measures accuracy and says nothing about calibration: a model can produce diverse, individually plausible samples whose spread bears no relation to its actual error, and still score well. Three families of diagnostics close the gap, and Section 3 implements them.

Proper scoring rules reward accuracy and calibration together and cannot be improved by misreporting uncertainty [10]. The continuous ranked probability score (CRPS) is the pointwise standard and reduces to mean absolute error for a deterministic forecast. Its multivariate generalization, the energy score [31], is the one that matters here: pointwise scores are blind to spatial correlation, so a model reproducing every marginal correctly while generating spatially incoherent fields scores well pointwise and badly under the energy score. Since PDE solutions are smooth and strongly correlated, that is exactly the failure worth catching.

Calibration diagnostics ask whether stated uncertainty matches observed error: coverage and the reliability curve compare nominal against empirical coverage; the rank histogram [12] plots where the truth falls among the members, flat meaning calibrated, U-shaped under-dispersed, dome-shaped over-dispersed; and the spread–skill ratio [8] compares spread against the ensemble mean's error, with a finite-ensemble correction $\sqrt{(K+1)/K}$ applied before it is read.

Decomposition splits predictive variance into an aleatoric part — one model's spread, the posterior width on a genuine inverse problem — and an epistemic part, the disagreement between independently trained models. That is what makes Remark 5 operational: on a forward problem the aleatoric component ought to be zero, so any observed spread is misattributed epistemic error.

3 FlowPDE

Section 2 described a design space. This section describes the software that makes it navigable. `flowpde` is a PyTorch library for training and sampling conditional continuous-time flows on PDE data. Its pieces are *orthogonal*: the dynamics, the objective that fits them, the network that parametrizes them, the data, and the integrator that samples them are five separable concerns, each replaceable without touching the others. Listings are quoted from the source, abridged, with elisions marked.

3.1 Library overview

The design goal is best stated by example. Listing 1 is a complete training and sampling run on the 2D Darcy forward problem. There is no configuration file, no framework, and no per-problem training script.

```
from flowpde import NeuralODEFlow, FlowMatchingObjective, UNet, Trainer
from flowpde.datasets.exponax import DarcyGenerator
from flowpde.datasets import FieldNormalizer
from flowpde.trainers import FlowEvaluator

# 1. Data: (kappa, f) -> u on a 64x64 grid.
gen = DarcyGenerator(num_points=64, kappa_alpha=2.0, kappa_tau=3.0)
train_ds = gen.generate(num_samples=1000, seed=0, problem='forward')
val_ds = gen.generate(num_samples=200, seed=1, problem='forward')

# 2. Normalization: fit on train, share the SAME instance with val.
normalizer = FieldNormalizer.from_dataset(train_ds)
train_ds.set_normalizer(normalizer); val_ds.set_normalizer(normalizer)

# 3. Velocity field, dynamics, objective.
model = UNet(spatial_dim=2, spatial_size=64,
             solution_channels=1, condition_channels=2, base_channels=64)
flow = NeuralODEFlow(model, target_key='target', condition_key='input')
objective = FlowMatchingObjective(flow, path='linear', time_sampler='uniform')

# 4. Train, selecting on sampled error rather than on the loss.
evaluator = FlowEvaluator(objective, val_loader, normalizer=normalizer,
                          target_fields=val_ds.target_fields)
trainer = Trainer(objective, torch.optim.Adam(model.parameters(), lr=1e-3),
                  ema_decay=0.999, validator=evaluator, monitor='rel_l2',
                  checkpoint_extra={'normalizer_state': normalizer.state_dict()})
trainer.train(train_loader, epochs=200, save_dir='runs/darcy')

# 5. Sample: n_steps is the accuracy/cost dial of Section 2.1.3.
u = objective.sample(condition=batch['input'], n_steps=50, solver='euler')
```

Listing 1: A complete run. Every line is a modelling choice; none is boilerplate.

The central separation. The key structural fact is visible in step 3: `NeuralODEFlow` and `FlowMatchingObjective` are two objects, not one. This is Remark 1 made concrete. The flow owns the *dynamics* — the velocity field, integration, and exact log-likelihood by (4). The objective owns the *training procedure*. Swapping `FlowMatchingObjective(flow)` for `MaximumLikelihoodObjective(flow)` changes how the same field is fitted and nothing else: same model, same data, same normalization, same sampler. The comparison between simulation-free and simulation-based training is therefore a controlled experiment rather than a comparison between two codebases, which is what Section 4 exploits. The converse discipline is enforced too: no training logic in the flow, no dynamics in an objective.

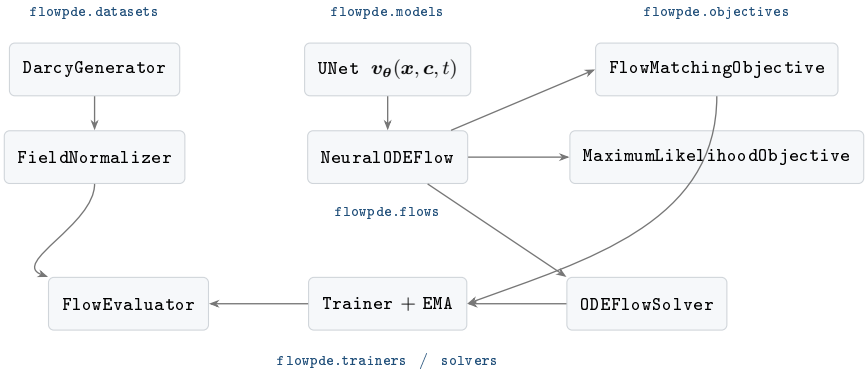

Fig. 1. Module dependencies. Arrows point from a component to the one that consumes it. The flow depends on a network and nothing else; the objectives depend on the flow; the trainer depends only on an object exposing `compute_loss` and `model`, so it is agnostic to which objective it is given. No arrow points into `flowpde.core.config`, which is optional throughout.

3.2 Modules

`flowpde.core`. Three abstract bases — `BaseFlow`, `BaseSolver`, `BaseConditioner` — plus an optional configuration layer that the core never imports. `BaseFlow`'s most consequential contribution is a tensor contract rather than an abstract method: `_extract_target_condition` flattens target and condition to (B, d) before anything else sees them. A field on a 64×64 grid reaches the objective as a vector of length 4096, and the convolutional backbones reshape it back internally. The benefit is that flow, objective, path, coupling and solver are written once in $\mathbb{R}^d$ with no spatial special-casing; the cost is that shapes must be declared to the backbone at construction, which Remark 7 returns to.

`BaseConditioner` abstracts *how* the condition enters the network. `ConcatConditioner` concatenates $\boldsymbol{c}$ to $\boldsymbol{x}$ along channels; `FiLMConditioner` modulates features by $\gamma(\boldsymbol{c}) \odot h + \beta(\boldsymbol{c})$; `NullConditioner` discards the condition. The last is not a placeholder but the control variant — a model that cannot see the PDE parameters establishes the floor any conditional model must clear.

`flowpde.datasets`. Data generation runs through Exponax [16] on JAX [2]. `PoissonGenerator` solves $\nabla^2 u = f$ spectrally. `BurgersGenerator` advances an initial condition with an exponential time-differencing stepper, and can draw a per-sample viscosity $\nu \sim$ LogUniform, though the experiments of Section 4 fix $\nu = 10^{-2}$. `DarcyGenerator` solves $-\nabla \cdot (\kappa \nabla u) = f$ with second-order finite differences and a fixed-step conjugate gradient written through `jax.lax.scan`, so the pipeline is `jax.vmap`-able with no Python loop over samples; κ is a log-normal Gaussian random field,

$$\kappa = \exp(c\, g), \qquad g \sim \mathcal{N}\big(0, (\tau^2 I - \Delta)^{-\alpha}\big), \tag{18}$$

which guarantees $\kappa > 0$ everywhere and reproduces the FNO Darcy benchmark at $\alpha = 2$, $\tau = 3$ [19].

Every generator takes `problem='forward'` or `'inverse'`, which swaps input and target, and the inverse direction activates additive observation noise (`obs_noise_std`) and random spatial masking (`obs_mask_fraction`). Together these implement (16) and make posterior degeneracy a controlled variable. The mask is *kept* rather than discarded, because a masked observation is otherwise ambiguous: a zero may mean "the field is zero here" or "nothing was measured here". It is appended as an extra conditioning channel, so the model must be built with `condition_channels` matching `sample['input'].shape[0]`.

Normalization. Flow matching transports $\mathcal{N}(\mathbf{0}, I)$ to the data, so a target with non-unit scale makes the velocity regression (10) badly conditioned. `FieldNormalizer` standardizes each field, and two decisions matter. Statistics are keyed by *raw field name* — `'source'`, `'solution'`, `'kappa'` — not by the role the field plays, which is why one normalizer stays valid across `'forward'` and `'inverse'` where the roles exchange but the fields do not. And a field with no registered statistics passes through untouched, which keeps `obs_mask` binary instead of standardizing an indicator into meaninglessness. Metrics are consequently reported in physical units: predictions are mapped back with `denormalize_channels`, and `FlowEvaluator` refuses to run with a normalizer but no `target_fields` rather than silently scoring in standardized units.

Remark 6 (Never refit on validation)**.** *`FieldNormalizer.from_dataset(train_ds)` is called once and the resulting* instance *is shared with validation and test. Fitting a second normalizer on held-out data leaks its statistics into the evaluation. The API makes the correct usage the short one; it cannot make the incorrect one impossible.*

`flowpde.models`. Four backbones share one signature, `forward(x, f, t)`, and reshape internally because the flow flattens. `MLP` is fully connected, for problems where spatial structure carries no information. `ConvNet` is a stack of time-conditioned residual convolutions at fixed resolution. `ResNet` [13] deepens that stack. `UNet` [28] adds encoder–decoder skips with optional bottleneck attention and a depth derived from the resolution, $\min(5, \lfloor \log_2 S \rfloor - 2)$, so downsampling stops at four to eight pixels rather than one.

Two details recur. Time enters through a sinusoidal embedding added residually *inside each block*, so every level is time-aware. And the output projection is *zero-initialized*, so a freshly constructed backbone predicts exactly $\boldsymbol{v}_{\boldsymbol{\theta}} \equiv \mathbf{0}$ — standard for flow models, since the flow starts as the identity, but it means a test that perturbs the input of a fresh model and expects the output to change will fail until `output_proj.weight` is perturbed first.

`flowpde.flows` and `flowpde.objectives`. `NeuralODEFlow` implements Section 2.1.1: it integrates (1) through `torchdiffeq` and computes exact conditional log-likelihood by augmenting the state with $\log p$, using either an exact trace at d Jacobian-vector products or the Hutchinson estimator (6) at one. One interaction deserves comment: the likelihood needs higher-order derivatives through the trace estimator, and the adjoint method's custom backward solve does not suit that graph, so the implementation disables the adjoint when `compute_logp` is set.

`FlowMatchingObjective` composes the four components of Section 2.1.2, each accepted as either a registry name or a constructed instance — which is what makes the API config-friendly without making configuration a dependency. Its entire training step is Listing 2, and it corresponds to (10) line by line.

```python
def compute_loss(self, batch, target_key=None, condition_key=None, **kwargs):
    x_1, condition = self.flow._extract_target_condition(batch, ...)

    # Passing the batch lets BatchSource return the precomputed x_0
    # that reflow depends on, instead of drawing fresh noise.
    x_0 = self.sample_base_distribution(x_1.shape, self.model_device, batch)

    # A source carrying its own pairing already fixes which x_0 goes with
    # which x_1, so re-coupling here would destroy it.
    if not self._source_defines_pairing(batch):
        x_0, x_1 = self.coupling(x_0, x_1)

    t = self.time_sampler(batch_size, self.model_device)
    x_t, v_target = self.path(x_0, x_1, t) # interpolate + velocity
    v_pred = self.model(x_t, condition, t)
    return F.mse_loss(v_pred, v_target)
```

Listing 2: `FlowMatchingObjective.compute_loss` — (10) in eight lines.

Table 1 lists what may be substituted at each slot. The two guarded branches are the correctness content of Remarks 2 and 4: the path returns interpolant and velocity *together*, so they cannot disagree, and the coupling is skipped when the source already defines the pairing, so minibatch-OT cannot silently reorder reflow's pairs. `BatchSource` is `strict=True` by default, so a missing key raises rather than resampling.

`flowpde.solvers` and `flowpde.trainers`. `ODEFlowSolver` wraps `torchdiffeq`, exposing fixed-step methods (`euler`, `midpoint`, `rk4`) and adaptive ones (`dopri5` [6], `tsit5`, others). The condition is held fixed while $\boldsymbol{x}$ evolves, implementing the remark that (1) moves the state and not the conditioning variable. Solver and step count are chosen at call time on trained weights, which is what makes the accuracy-against-cost curve of Section 4.4 measurable without retraining.

`Trainer` accepts any object exposing `compute_loss(batch)` and a `.model`, which is why it depends on neither objective specifically. Two of its features are less standard and more important.

Exponential moving averaging. The flow-matching gradient is noisy in a structural way: by Theorem 1 the regression target is only conditionally determined by the network's input, so iterates bounce around the

Slot	Options	What it controls
Path	`linear`, `ot_conditional`	the interpolant and its velocity
Time	`uniform`, `logit_normal`, `beta`, `truncated`	where capacity is spent in t
Coupling	`independent`, `minibatch_ot`	which $\boldsymbol{x}_0$ meets which $\boldsymbol{x}_1$
Source	`gaussian`, `batch`	where trajectories start

Table 1. The four flow-matching slots. Each accepts a registry name or an instance, so the variants of Section 2.1.2 — standard flow matching, rectified flow, OT-CFM, minibatch-OT coupling — are configurations rather than subclasses.

optimum rather than settling. Averaging suppresses that, and the averaged weights are what is evaluated and shipped. A warmup ramp, $\min(d, (1+n)/(10+n))$, removes the need for a "start averaging at step N" threshold.

Selecting on sampled error, not on the loss. This is the most consequential choice in the module. As Section 2.1.2 explained, the flow-matching loss has a large irreducible floor, and two checkpoints can differ substantially in sample quality while their losses differ in the fourth decimal. `FlowEvaluator` therefore integrates the ODE, denormalizes, and scores against ground truth, and `Trainer` selects on that. Evaluation noise uses a *fixed seed*, so epoch-to-epoch differences reflect the model rather than the draw. With `ensemble_size > 1` it scores the ensemble mean and reports the spread, which is where the diagnostics of Section 2.2.2 attach: `flowpde.utils.uq_metrics` implements coverage, reliability curves, rank histograms, spread–skill with the $\sqrt{(K+1)/K}$ correction, CRPS, the energy score, and the aleatoric/epistemic decomposition.

Reflow, and what enforcing its invariant bought. Remark 4 states the correctness condition: training must consume the same $\boldsymbol{x}_0$ that generated each target, and `BatchSource` is `strict=True` so a missing key raises rather than silently resampling. That machinery was exercised on the Burgers ConvNet at three seeds, two reflow iterations, against a deliberately broken control that resamples $\boldsymbol{x}_0$ independently — the bug the invariant exists to prevent. Two results, both negative, are worth recording. The correct procedure does straighten, from 8.23 to 5.18, but *so does the broken one*, from 8.23 to 5.81; retraining on the model's own samples apparently straightens on its own, so straightening is not by itself evidence that the pairing was honoured. And neither arm improved few-step accuracy: one-step relative L^2 moved from 0.0996 to 0.1048 (paired) and 0.0999 (decoupled), and error rose at every larger budget. On this problem the baseline was already accurate at eight evaluations, leaving reflow little to recover and two iterations of retraining to lose. The invariant is still a correctness condition and is still enforced structurally; what the experiment shows is that its *observable* consequence is smaller and less diagnostic than the literature's framing suggests.

Remark 7 (A known rough edge)**.** *The flattening contract means the flow must be told the target dimension when it differs from the condition dimension — as it does for Darcy, where the condition is $[\kappa, f]$ on two channels and the target is u on one. At present `flow._target_dim` is set as a side effect of `compute_loss`, so a checkpoint loaded fresh for inference has never had it set and `sample()` silently falls back to the condition dimension. The workaround is to pass `target_shape=` explicitly; the fix is to accept the shape at construction. It is recorded here because it is exactly the class of defect a report describing its own software should disclose.*

3.3 Installation and testing

`flowpde` targets Python 3.11 and depends on `torch` [24], `torchdiffeq`, and `jax`/`exponax` [2, 16]. The environment is managed with `uv`: `uv venv && uv sync`, then `uv run -m pytest`.

The suite covers components, models, solver, normalization, trainer, EMA, evaluation, straightness, reflow and the UQ metrics. Its convention is to prefer *analytically known* expectations over regression baselines wherever one exists: path velocities against finite differences of the interpolant, the solver against $\mathrm{d}\boldsymbol{x}/\mathrm{d}t = -\boldsymbol{x}$, the straightness of $\boldsymbol{v} = 2t\boldsymbol{c}$ against its exact value of $1/3$, and the EMA recursion against hand-computed arithmetic. Tests invoking the Exponax solvers are marked `slow`.

4 Numerical experiments

Every comparison below varies *one* axis with everything else held fixed — data, splits, normalization, optimizer, schedule, averaging, sampler and evaluation seed — so a difference between two numbers is attributable to the thing named in the column heading. Where that discipline is imperfect it is stated; Section 4.10 collects the cases.

Two earlier commitments constrain what may be claimed. Remark 5 forbids reading sampler spread on a forward problem as uncertainty, so the forward experiments are reported as verification and as measurements of cost. And Section 2.2.2 committed to scoring a distribution as a distribution, which is why Section 4.7 reports proper scoring rules rather than relative L^2 alone.

4.1 Setup

Benchmarks. Three PDE families are used. *Poisson*, $\nabla^2 u = f$, is learned forward at 64×64 from a random sine source, at two spectral widths (below). *Burgers* in 1D maps an initial condition to the state at $T = 0.4$ on 64 points, at fixed viscosity $\nu = 10^{-2}$. *Darcy* in 2D, $-\nabla\cdot(\kappa\nabla u) = f$ on $[0,1]^2$ at 64×64, carries most of the experiments: κ is the log-normal field (18) at $\alpha = 2$, $\tau = 3$ — the FNO benchmark parameters [19] — the source is a random Fourier field with cutoff 8, and the linear system is solved by 500 conjugate-gradient iterations.

Why the inverse direction is harder. The forward Poisson map multiplies by $-|k|^{-2}$ in Fourier space. Figure 2 measures that gain over a dataset and finds it follows $|k|^{-2}$ closely. The consequence is asymmetric: splitting a source–solution pair at $|k| = 5$ leaves 36% of the source's energy above the cut and 0.6% of the solution's. The forward map discards the fine structure of f, and an inverse model must reconstruct it from a field that barely contains it. This is why (17) has genuine width.

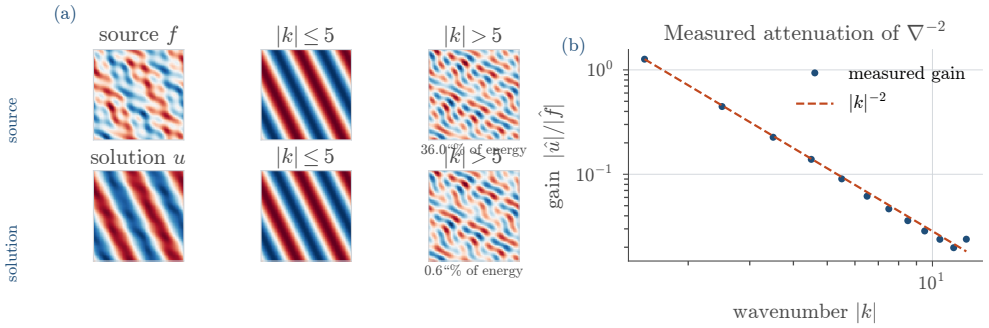

Fig. 2. (a) A Poisson source and solution split at $|k| = 5$: above the cut sits 36% of the source's energy and 0.6% of the solution's. **(b)** Measured spectral gain against $|k|^{-2}$. Both use a *widened* source, since the generator's default carries no energy above $|k| = 3$ — itself a reason to distrust errors measured on it.

How much of an error is a property of the benchmark. That last remark is testable, and E8 tests it. The Poisson source is a random sine series with n terms at wavenumbers up to $k_{\max}$; the generator's default, $n = k_{\max} = 3$, spans a nine-dimensional subspace. Training the same two backbones on that default and on a widened $n = k_{\max} = 8$ moves relative L^2 from 0.054 to 0.078 for the ConvNet and from 0.061 to 0.066 for the UNet — a 43% increase for the better model from widening the source alone. An operator-learning number is therefore a joint statement about the method and the distribution it was measured on, and the harder regime is the one quoted whenever a choice arises below.

What is held fixed. Every run trains a conditional `NeuralODEFlow` on the standard flow-matching configuration — linear path, uniform time sampler, independent coupling, Gaussian source — with AdamW at 2×10^{-4}, cosine decay, gradient clipping at 1.0, and EMA at $\rho = 0.999$. Splits are 3000/200/200 at fixed generator seeds, so every variant sees byte-identical data. One `FieldNormalizer` is fitted on train and shared (Remark 6). Selection is on sampled validation error at a fixed seed, never on the training loss. Table 2 records what each experiment varies.

Metrics. Errors are computed in physical units and per test sample. Relative L^2 is $\|\hat{\boldsymbol{u}} - \boldsymbol{u}\|_2 / \|\boldsymbol{u}\|_2$ and relative H^1 the same ratio in the discrete H^1 norm. Also reported are $\varepsilon_\infty = \max_x |\hat{u} - u|/\mathrm{std}(u)$, normalized by a *spread* rather than a range and so legitimately above one, and errors restricted to the 20% of the domain with the largest $|f|$ or κ.

Remark 8 (Relative L^2 near one is not a small number)**.** *Several inverse models below score relative $L^2 \approx 1$. That is roughly what using none of the conditioning information achieves, and there are two such levels. A* deterministic *model returning the mean field scores exactly* 1*. A* generative *model drawing from the marginal returns an independent sample, so for a zero-mean field $\mathbb{E}\,\|\hat{\boldsymbol{u}} - \boldsymbol{u}\|^2 = 2\mathbb{E}\,\|\boldsymbol{u}\|^2$ and the score is about $\sqrt{2}$. The gap is not a defect: it is the price of returning a sharp admissible field instead of a*

	Problem	Varied	Levels	Epochs
E1	Darcy 2D, fwd.	conditioning	null, concat	500
E2	Burgers 1D, fwd.†	backbone	ConvNet, ResNet, UNet, UNet$^-$	500
E3	Darcy 2D, fwd.	backbone	ConvNet, ResNet, UNet, UNet$^-$	500
E4	Darcy 2D, fwd. $16^{2\dagger}$	objective	flow matching, max. likelihood	50
E5	Darcy 2D, inverse	conditioning, target	null/concat $\times$ $\kappa/(\kappa, f)$	100, 500
E6	Darcy 2D, inverse	backbone	ConvNet, ResNet, UNet, UNet$^-$	500
E7	Burgers 1D, fwd.†	path, schedule, coupling	6 configurations	500
E8	Poisson 2D, fwd.	backbone, source spectrum	2×2	500
E9	Darcy 2D, inverse	observed fraction, noise	$4 + 4$ levels	100
E10	Darcy 2D, inverse	sampler: pCN reference	4 obs. $\times$ 2 chains	—
Re-evaluations on finished weights — no retraining				
R1	Burgers, Darcy, Poisson	solver, step count	4 solvers, 8 step counts	—
R2	Burgers, Darcy	straightness mode	trajectory, interpolant	—
R3	Darcy, both directions	ensemble size	$K = 32$	—

Table 2. The experiment matrix. UNet$^-$ is the UNet without attention. † marks the experiments replicated at three matched seeds (42, 1042, 2042) per cell; the rest are single runs. R1–R3 are measurements on finished weights, possible without retraining because sampling cost, straightness and ensemble size are inference-time choices.

blurred average, and it is why a single draw must not be scored as a point estimate. Section 4.7 therefore keeps an explicitly unconditional model in every comparison.

Hardware. E1–E9 ran on a CUDA device, R1–R3 afterwards on Apple silicon. Error metrics are deterministic given the seed, but wall-clock numbers from the two machines are compared only within a table.

4.2 Does the condition reach the velocity field?

A conditional flow can fail invisibly: if the conditioning tensor never influences the velocity, the model still learns a good *unconditional* generator of Darcy solutions, its loss still decreases, and its samples still look like plausible fields. Only sampled error against matched ground truth detects it. E1 replaces `ConcatConditioner` with `NullConditioner` and changes nothing else.

On the same 200 test samples the conditioned model is better on *every one*, with a median per-sample ratio of 17.2; median relative L^2 falls from 1.30 to 0.092 and mean relative H^1 from 2.27 to 0.133. Both loss curves converge and look healthy. This is the concrete argument for selecting on `FlowEvaluator` rather than on the loss. It also calibrates the rest of the section: the null model is an honest unconditional sampler, so 1.30 is what learning nothing looks like here.

4.3 Backbone

E2 and E3 vary only the network parametrizing $\boldsymbol{v}_{\boldsymbol{\theta}}$. Figure 3 plots accuracy against parameter count, and the smallest backbone wins on both problems. On Burgers the ConvNet at 0.31M parameters reaches relative L^2 0.021 against the UNet's 0.266 at 2.34M — a factor of 12.4 in the wrong direction for capacity. On Darcy the ConvNet at 0.73M reaches 0.073 against 0.128 at 5.70M, and is better on 176 of 200 individual test samples. Attention changes nothing: UNet and UNet$^-$ agree to within 1.5% on Darcy, and on Burgers to within their own seed noise (below).

The obvious objection is that a single run per cell measures seeds rather than architectures. E2 was therefore rerun at three *matched* seeds per backbone — the same set for every variant, so none is advantaged by its draw — giving ConvNet 0.0214 ± 0.0003, ResNet 0.1420 ± 0.0026, UNet 0.2657 ± 0.0017 and UNet$^-$ 0.2660 ± 0.0013. A seed spread of 0.5–1.8% against a 12.4$\times$ gap settles the ordering, and settles the attention question too: the two UNets differ by 0.1% between seed means, five times *less* than the seed noise. The Darcy replication was started and abandoned, so E3 keeps the weaker guarantee.

The ordering on Darcy is stable across aggregate norms and both region-restricted errors, so no backbone trades bulk accuracy for accuracy where κ is large. On Burgers the aggregate hides a spectral failure common to all four: above $|k| = 13$ every model is wrong by a factor between 48 and 400, in a band carrying too little energy to register in relative L^2 — the observation motivating PDE-Refiner [22], here measured rather than assumed.

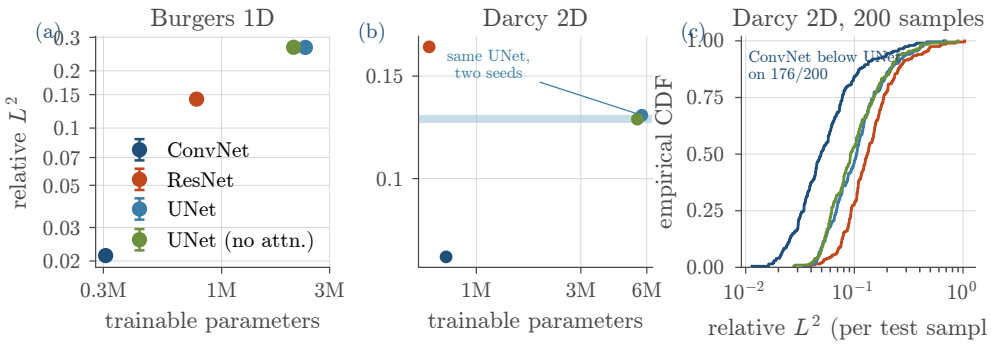

Fig. 3. (a, b) Accuracy against parameters; the ordering is inverted with respect to capacity. Error bars in (a) are the standard deviation over three matched seeds, and are smaller than the markers. The band in (b) spans two runs of the *identical* UNet at different seeds — the only replicate available on Darcy. **(c)** Per-sample errors on Darcy.

4.4 Accuracy against sampling cost

Sampling is an ODE solve, so the evaluations spent per solution are chosen at call time on frozen weights. R1 sweeps that. Error is plotted against *function evaluations* rather than steps, since one RK4 step costs four and a per-step comparison would credit RK4 with a factor of four it has not earned. Three results follow, none of them the expected one.

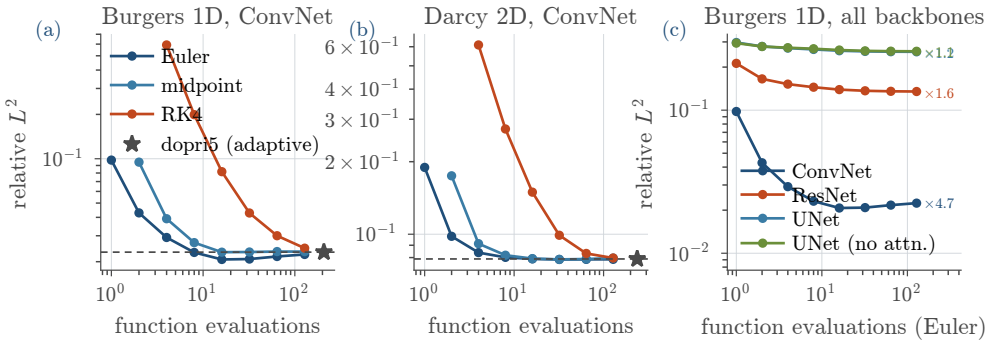

Fig. 4. Accuracy against cost on frozen weights. **(a, b)** The best backbone per problem; the star and dashed line mark the converged solution of the learned ODE. **(c)** Euler on every Burgers backbone. The *slope* is the diagnostic: steep means discretization error dominates, flat means the field is the limitation.

Euler beats the higher-order solvers at matched cost. On Burgers with the ConvNet, Euler and RK4 at the same budget differ by 8.6× at 8 evaluations (0.0231 against 0.1992), 3.9× at 16 and 2.1× at 32. On Darcy the ratios are 3.4, 1.9 and 1.3, and every backbone on both problems has RK4 at or behind Euler below 64 evaluations. The mechanism: a high-order tableau buys accuracy by evaluating the field *off* the trajectory — a single RK4 step across $[0, 1]$ probes $\boldsymbol{v}_{\boldsymbol{\theta}}$ at $\boldsymbol{x} + h\boldsymbol{k}_3$, far from anything seen in training — and a learned field has no accuracy guarantee there. Classical order counts terms in a Taylor expansion of the true field; here the field itself is the approximation. The gap closes where model error dominates: on the three inaccurate Burgers backbones, Euler and RK4 agree within a few per cent at every budget.

The slope separates model error from discretization error. The ConvNet's Burgers error falls by 4.7× from one evaluation to its best; the ResNet's by 1.6 and the UNet's by only 1.2 out to 128. A flat curve means the integrator was never the binding constraint. The ordering is the same on Darcy though compressed (2.4, 2.0, 1.5, 1.4), and runs parallel to accuracy — the most accurate model has most to gain from a finer discretization. The sweep is therefore a cheap diagnostic, not just a cost table.

Converging the ODE is not the goal. The ConvNet on Burgers is best at 0.0207 with sixteen Euler evaluations. Integrating the same weights to convergence (`dopri5`, 208 evaluations) gives 0.0232 — 11% *worse*. Coarse Euler lands closer to the data than the exact solution of the learned ODE, because its truncation error partially cancels the model's own. The effect recurs but is not systematic: on Darcy the ResNet gains 6% this way and the ConvNet 0.5%, while the two UNets match their converged error to four

decimals. What can be said is the negative claim, and it is enough: the converged solution of the learned ODE has no special status, and solver tolerance is not a knob to tighten until the answer stops changing. Reporting accuracy "at 50 steps" quotes one point on a non-monotone, model-dependent curve.

Against the solver it replaces. Both sides timed on the same device (CPU, eight threads), batched over the same 200 conditions: the reference conjugate-gradient solve costs 9.7 ms per solution, the ConvNet surrogate 18.7 ms at one evaluation and 977 ms at fifty. The surrogate is therefore *never* cheaper — 1.9× the solver at its cheapest and roughly 65× where it is most accurate. On a linear elliptic problem at 64×64, against a well-implemented batched conjugate gradient, a neural surrogate of this size does not pay for itself. The forward experiments demonstrate that the operator can be learned and measure what learning it costs; they are not a speed argument. Two things follow rather than being contradicted: the solver's cost grows with discretization and nonlinearity while the network's does not, and the case for the method was never the forward direction. In Section 4.7 the classical alternative is not one solve but Markov chain Monte Carlo, needing thousands of these solves *per observation*; against that, one amortized ODE solve is the offline/online split doing its job.

Straightness, and why this was the wrong experiment for it. R2 measured straightness on the same checkpoints and found it barely moves across backbones — 0.047 to 0.056 on Burgers — against accuracy differences of an order of magnitude. Path geometry belongs to the *objective*, and all eight models share one: changing the network changes how well the field is fitted, not which transport it is fitted to. Section 4.5 varies the axis that does move it.

4.5 The four components

Section 3.1 claimed that the published flow-matching variants are *configurations* of one objective — an interpolation path, a time sampler, a noise–data coupling and a source distribution — rather than subclasses. E7 is the experiment that claim owes: six configurations reached by changing registry strings, on one Burgers ConvNet, at three matched seeds each, with everything else byte-identical. It is also the experiment Section 4.4 said was missing.

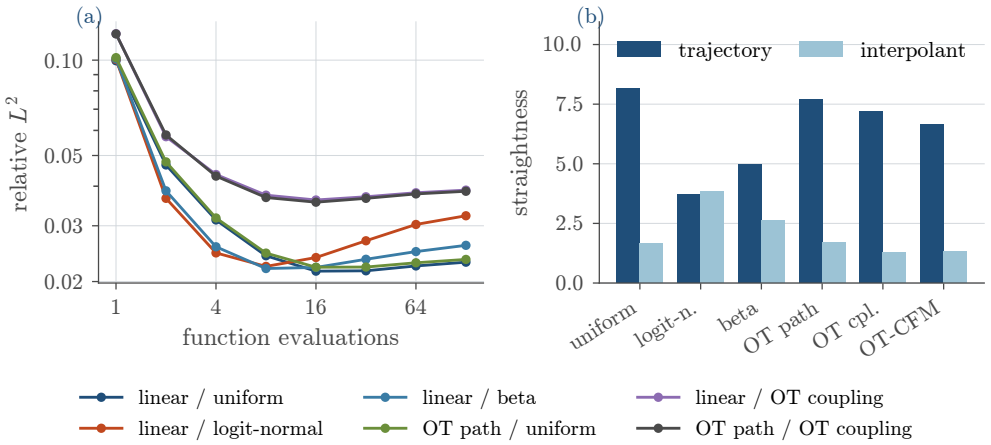

Fig. 5. **(a)** Error against function evaluations, Euler, averaged over three seeds. The curves cross: the configuration that wins below eight evaluations loses above sixteen. **(b)** The two straightness definitions disagree about which configuration is straight, and only the trajectory one predicts (a).

The time schedule is what straightens trajectories. Logit-normal sampling [7] cuts trajectory straightness from 8.17 to 3.70 and the beta sampler to 4.95, while both optimal-transport variants leave it between 6.6 and 7.7. The payoff is where straightness is supposed to pay: at two function evaluations logit-normal reaches 0.0366 against the uniform baseline's 0.0467, and at four 0.0246 against 0.0313 — 22% better in both cases, against a seed spread of 1.6%. Concentrating the sampled times near the endpoints fits the field better where a coarse Euler step needs it.

And it costs converged accuracy. The same variant is the *worst* non-OT configuration once the budget is adequate: 0.0280 at convergence against the baseline's 0.0214, a 31% loss. The curves in Figure 5(a) cross near twelve evaluations. Straightening is not a free improvement to the model; it moves

error from the few-step regime into the converged one, and which side of the crossing a practitioner sits on decides whether it is worth doing.

Optimal-transport coupling did not pay, and the two straightness definitions explain why. Minibatch-OT coupling and OT-CFM are worse than the baseline at *every* budget — 0.0375 and 0.0372 converged against 0.0214, and 0.121 against 0.0996 at a single evaluation. This contradicts the expectation the experiment was set up to test. The diagnostic in Figure 5(b) says what happened: the OT coupling produces by far the straightest *interpolant* (1.28 against 1.66) while leaving the model's own *trajectories* as curved as the baseline's. It straightened the training target, not the learned dynamics. This is the concrete reason the library implements both modes of `estimate_straightness`, and why Definition 1 is stated on trajectories: a report using interpolant straightness alone would have concluded that OT coupling worked. With 64 samples per minibatch the assignment is also a poor approximation to the true OT map, which is the standard caveat [32] and the most likely mechanism.

What the experiment demonstrates about the code. Six configurations, one class, no subclassing; the OT path and the OT coupling are separate strings, which is why panel (b) can attribute the effect to the coupling rather than to "OT" as a bundle. The cost of the whole matrix was 18 runs at roughly 290 seconds each.

4.6 Objective: simulation-free against simulation-based

E4 is the experiment the flow/objective separation was built to allow. One flow, one UNet of 40,777 parameters, one dataset, one sampler; the only difference is whether the weights are fitted by `FlowMatchingObjective` or by `MaximumLikelihoodObjective`, which integrates the augmented dynamics with a Hutchinson trace estimator at every step. The problem is deliberately small — Darcy at 16×16 — because the simulation-based route is not affordable at 64×64.

Objective	s / epoch	peak MB	rel. L^2	rel. H^1	NLL / dim
Flow matching	**1.92**	**18.6**	$\mathbf{0.610 \pm 0.026}$	$\mathbf{0.656 \pm 0.029}$	-1.214 ± 0.062
Maximum likelihood, 1 probe	135.2	166.3	1.064 ± 0.419	1.016 ± 0.386	$\mathbf{-1.856 \pm 0.174}$
Max. likelihood, 2 and 4 probes	did not complete				

Table 3. Training objective on an identical conditional flow, Darcy 16^2, 50 epochs, mean $\pm$ standard deviation over three matched seeds. Each objective wins on the metric it optimizes — but only one of them wins reproducibly.

The outcome is unusually clean: *each objective wins on its own metric.* Maximum likelihood reaches a test negative log-likelihood of -1.86 per dimension against -1.21, which is what it is built for. Flow matching produces samples at relative L^2 0.610 against 1.064 — the latter, by Remark 8, no better than returning the mean field — and does so at 1.92 seconds per epoch against 135.2, a factor of 70, using 18.6 MB of peak memory against 166.3.

The replication adds a second axis to the comparison. Because this cell is cheap on one side, it was run at three matched seeds, and the two objectives differ in *stability* as sharply as in cost. Flow matching varies by 4.2% across seeds; maximum likelihood varies by 39.3%, its three runs scoring 0.745, 0.909 and 1.538. The best MLE seed is a respectable model and the worst is worse than the prior, from nothing but the draw. The stochastic trace estimator injects gradient noise that the simulation-free objective does not have, and at this budget it is not averaged out. A single-run comparison would have reported whichever of those three numbers it happened to draw.

The honest reading is therefore narrower than "flow matching is better". At a fixed small budget on this problem, the simulation-free objective buys sample quality and reproducibility, and the simulation-based one buys density estimates, at two orders of magnitude more compute. The two larger trace-probe variants were launched and never finished; they are listed as not completed rather than omitted, since at 135 seconds per epoch for a single probe on a 16×16 problem, the fact that more probes were unaffordable is itself the measurement.

4.7 Inverse problems and calibration

E5 moves to the setting the generative argument was made for. The observation model is (16) with $\mathcal{O}$ a random restriction keeping 15% of the grid and $\sigma = 2 \times 10^{-4}$; the target is either κ alone, conditioning

on (u_{obs}, f, m), or the pair (κ, f) jointly, conditioning on (u_{obs}, m). Every cell reported here uses 5000 training samples, with the null and concat conditioners.

The unconditional model is the measuring stick. A null-conditioned model on an inverse problem is a generative prior over κ: it produces admissible fields unrelated to the observation. It scores single-draw relative L^2 of 1.069 on the κ target and 1.104 on the joint one — against an ensemble spread of 1.46 on the latter, wider than the error it is trying to cover. That is what "no information was used" looks like here, and why Remark 8 had to come first. Every comparison below keeps one in the table.

Training budget, not training-set size, moved these models. The inverse cells were run at two budgets, and disentangling them matters because the obvious reading is wrong. Holding the network, the data seeds and the observation model fixed, the κ target scores relative L^2 0.932 after 100 epochs on 5000 samples and 0.793 after 500 — while E6, at 3000 samples and 500 epochs, reaches 0.799. Five hundred epochs on 3000 samples therefore matches five hundred on 5000 to within 1%, whereas holding the data at 5000 and going from 100 to 500 epochs buys 15%. On this problem the binding constraint was optimization, not examples. The three runs come from different experiments and differ in training seed, so the claim is about the ordering of two large effects rather than a precise decomposition; the seed replication in Section 4.3 bounds the seed term near 1–2% on a problem where it was measured.

But relative L^2 is the wrong question. A single draw from a correct posterior *should* differ from the truth by roughly the posterior width. R3 therefore draws $K = 32$ samples per condition at a fixed seed and scores them properly (Table 4). As a by-product it re-derives the point metrics from its own draws, independently of the plotting script that produced the original files, reproducing the recorded win rates to within a few per cent.

Target	Cond.	CRPS	ES	cov. 90	spr/skl	$\rho_{e,s}$	L^2 (1)
Inverse, 5000 *samples,* 100 *epochs*							
$\kappa^{\ddagger}$	null	0.860	95.9	0.826	0.90	−0.89	1.069
$\kappa^{\ddagger}$	concat	**0.700**	**83.8**	0.838	0.92	−0.36	0.932
(κ, f)	null	0.631	100.2	0.825	0.92	−0.05	1.104
(κ, f)	concat	**0.471**	**84.8**	0.847	0.94	+0.02	0.952
Inverse, 5000 *samples,* 500 *epochs*							
κ	concat	0.747	94.0	0.703	0.47	+0.38	0.794
Forward, for contrast — true posterior is a Dirac							
u (ConvNet)	concat	$3.5{\cdot}10^{-5}$	0.003	0.959	1.16	+0.88	0.075
u (UNet)	concat	$1.5{\cdot}10^{-4}$	0.012	0.651	0.22	+0.77	0.136

Table 4. Distributional scores, $K = 32$ at a fixed seed, physical units. ES is the energy score; "spr/skl" is the spread–skill ratio after the $\sqrt{(K+1)/K}$ correction, 1 meaning calibrated and below 1 over-confident; $\rho_{e,s}$ is the rank correlation between predicted spread and actual error; L^2 (1) is a *single* draw's relative L^2. Rows marked ‡ are the reference cell of E9, scored on all 200 test conditions at ensemble seed 123; the rest are the R3 re-evaluation on 64 conditions at ensemble seed 7. Comparisons are made within a protocol.

Proper scoring rules separate more than the single draw does. In both matched null-versus-concat pairs the proper scores register a larger effect than the point metric. On the κ target a single draw says conditioning buys 12.8% (1.069 → 0.932) while CRPS says 18.6% (0.860 → 0.700); on the joint target the single draw says 13.8% and CRPS 25.4%. The conditioned models are not merely closer to the truth, they place a *better-shaped* distribution on it, which a metric scoring one draw against one truth cannot see. This is the concrete reason Section 2.2.2 insisted on scoring a distribution as a distribution: judged on relative L^2 alone, half of what conditioning bought here is invisible.

More *training* made the model sharper and over-confident. The one badly calibrated inverse model is the κ target trained for 500 epochs. Against the same configuration at 100 epochs its spread–skill ratio falls from 0.92 to 0.47 and its 90% coverage from 0.84 to 0.70, while its single-draw error improves from 0.932 to 0.794; its rank histogram in Figure 6(a) is the U shape of under-dispersion. The extra optimization made the posterior narrower than the evidence supports — a failure a point metric rewards and a proper scoring rule penalizes, and one that would have been read as a success from relative L^2 alone. The two runs are scored under the two protocols of Table 4, so the exact magnitudes carry that caveat; a change of this size is not a scoring-subset artefact, and the direction is unambiguous. It is a sharper

statement than the one this report previously made from the same weights, where the budget change was confounded with a change in training-set size.

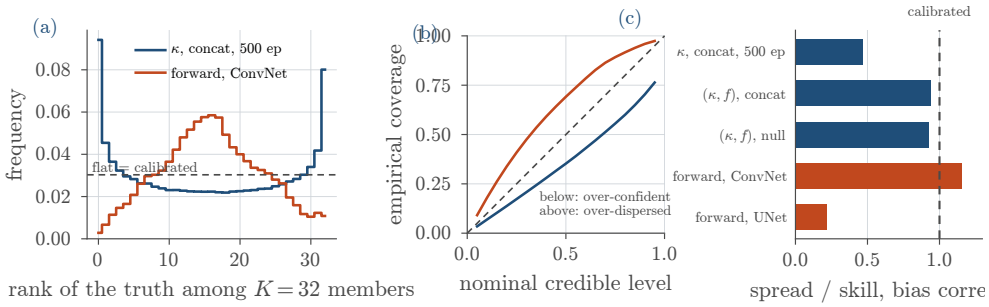

Fig. 6. Calibration, $K = 32$. **(a)** Rank histograms: the inverse model U-shaped (under-dispersed), the forward one dome-shaped. **(b)** Reliability against the diagonal. **(c)** Bias-corrected spread–skill; the two forward models, whose true posterior is identical and degenerate, disagree by a factor of five.

The forward experiment makes Remark 5 quantitative. Running the same suite on two *forward* models, where the true posterior is a point mass and the correct spread is zero, gives spread–skill ratios of 1.16 and 0.22. Two models approximating the same degenerate posterior report uncertainties differing by a factor of five, which is impossible if that uncertainty belongs to the problem and unsurprising if it belongs to the model. The practical corollary: averaging 32 draws cuts the ConvNet's relative L^2 from 0.075 to 0.035, because on a Dirac posterior all spread is error. One should always ensemble on a forward problem, and never call the result uncertainty quantification.

A refinement cuts against the simplest reading of Remark 5. Forward spread is not physical uncertainty, but it strongly predicts the model's own error: the rank correlation between per-sample spread and per-sample error is $+0.88$ and $+0.77$ for the forward models, against -0.06 to $+0.38$ for the inverse ones. Sampler spread is a usable *error estimate* exactly where it is not a posterior width, and a posterior width exactly where it does not track error — the opposite of what one would assume.

Where the data constrain the coefficient. Section 2.2.2 argued that κ enters only through the flux, so the observation constrains it where ∇u is large and barely at all where the solution is flat. Figure 7 tests that. Binning every pixel of every test condition by local $|\nabla u|$, the median posterior standard deviation falls from 0.907 in the lowest flux decile to 0.234 in the highest — a factor of 3.9, rank correlation -0.53 over all 260,000 pixels. The model was never told this; it is a property of the posterior it learned.

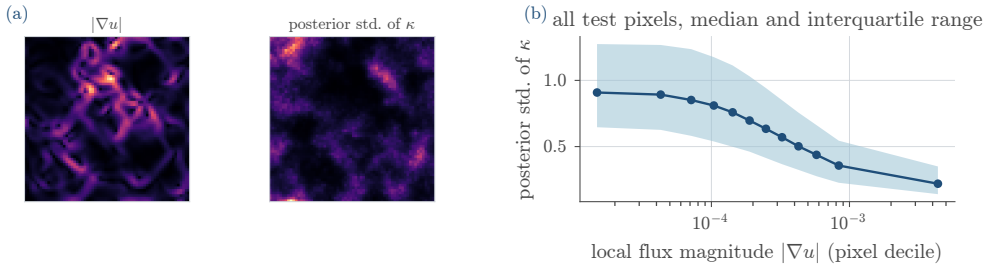

Fig. 7. Spatially varying identifiability. **(a)** $|\nabla u|$ and the posterior standard deviation of κ for one condition; the bright regions coincide. **(b)** All test pixels by flux decile: posterior width falls 3.9× from lowest to highest.

The backbone stops mattering. E6 repeats the E3 backbone comparison in the inverse direction, at 3000 samples and 500 epochs with the concat conditioner fixed. All four networks land between 0.788 and 0.820: ConvNet 0.788, UNet 0.799, UNet$^-$ 0.812, ResNet 0.820. Worst exceeds best by 4%; on the forward problem the same four networks span a factor of 2.3 (Section 4.3), and the ordering is preserved but flattened. Once the observation determines the target only up to a wide posterior, the approximation quality of the velocity field is no longer the binding constraint — the information content of the data is. The same conclusion arrives from the other direction in Section 4.4, where a flat error-against-budget curve means the integrator was never binding either. It also has a practical corollary: architecture search is a poor investment on these inverse problems, and Section 4.8 identifies what does move the numbers.

What this does and does not amount to. The inverse models are not accurate in the way the forward ones are, and nothing above claims they are. What is established is narrower and was the actual question: the samplers are approximately calibrated — all four 100-epoch inverse models have spread–skill between 0.90 and 0.94 — the conditional ones beat their unconditional counterparts on both proper scoring rules in every matched comparison, and the posterior carries spatial structure matching the physics. The missing piece is the aleatoric/epistemic decomposition, which needs several independently trained models per cell and so fell to the same single-run limitation as the rest of the inverse experiments.

4.8 Ill-posedness as a controlled variable

Everything above observes 15% of the grid at $\sigma = 2\times10^{-4}$, which makes the degree of posterior degeneracy an accident of one setting. Section 3.2 argued that it need not be: the mask fraction and the observation noise are generator parameters, so the amount of information in the observation is a dial. E9 turns it. Eight cells — four observed fractions at fixed noise, four noise levels at fixed masking, plus the unconditional model at the reference setting — each trained identically for 100 epochs on 5000 samples and each scored with the full $K = 32$ suite on all 200 test conditions.

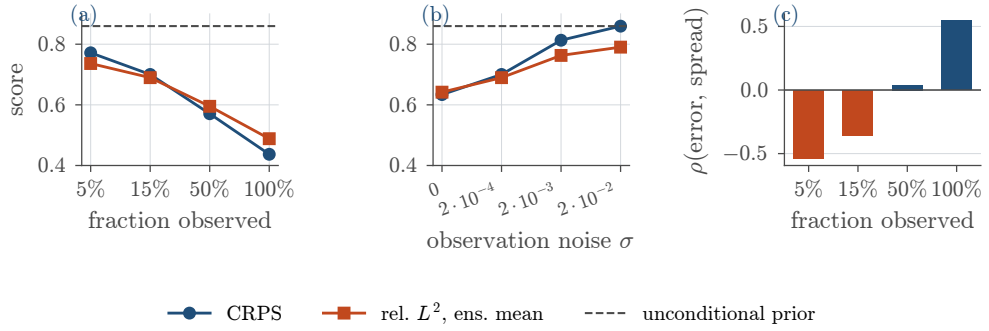

Fig. 8. Inverse Darcy, κ target, as the observation is degraded. **(a)** Observed fraction at $\sigma = 2\times10^{-4}$. **(b)** Noise at 15% observed. In both, the dashed line is the unconditional model: a curve reaching it is a model that has stopped using its data. **(c)** Rank correlation between per-sample ensemble spread and per-sample error, which changes sign across the sweep.

The dial works, and it spans the whole usable range. Going from 5% to full observation moves CRPS from 0.772 to 0.437 and ensemble-mean relative L^2 from 0.736 to 0.488. Going from noiseless to $\sigma = 2\times10^{-2}$ moves CRPS from 0.634 to 0.860. The upper end of that range is not an arbitrary bad number: 0.860 is the CRPS of the unconditional prior to three decimal places. At a hundredfold the reference noise the conditional model has been driven exactly onto the prior — it still produces admissible κ fields, and they carry no information about the observation. The claim that degeneracy is controlled is therefore demonstrated at both ends, and Remark 8's reference level is where a model lands when the data are destroyed rather than a hypothetical.

Where spread becomes an error estimate. Section 4.7 noted the awkward pattern that sampler spread predicts error on forward problems ($\rho = +0.88$) but not on inverse ones. The sweep shows it is not a forward/inverse dichotomy but a continuum in identifiability. At 5% observed the rank correlation between spread and error is -0.54; at 15%, -0.36; at 50%, $+0.04$; at full observation, $+0.54$. The sign flips as the problem becomes well determined. The negative branch has a mechanism: where the observation is uninformative the model reverts toward the prior, and prior draws are simultaneously *wide* and, for a κ field that happens to be near the prior mean, not especially wrong — so the widest predictions are not the worst ones. Spread is a usable error bar only once the data determine the answer, which is the regime in which one least needs it.

Calibration is the quantity that survives degradation. Across all eight cells the corrected spread–skill ratio stays between 0.88 and 0.93 and 90% coverage between 0.83 and 0.85, while CRPS nearly doubles. The models track how much they know even as what they know collapses. That is the property the method was adopted for, and it is worth separating from accuracy: a model can be uninformative and honest, and these are. The exception remains the over-trained cell of Section 4.7, which is sharper and less honest than any cell here.

4.9 The cost of a reference posterior

Everything above establishes that the inverse samplers are calibrated and use their data. None of it establishes that they approximate the *Bayesian* posterior, because no reference posterior was ever computed. E10 attempts one.

The natural algorithm is preconditioned Crank–Nicolson [5], and it is natural for a reason specific to this generator: κ is built by pushing white noise $\boldsymbol{\xi} \sim N(\mathbf{0}, \boldsymbol{I})$ through a fixed spectral filter (18), so sampling in $\boldsymbol{\xi}$ rather than in κ makes the prior exactly Gaussian. It then cancels from the acceptance ratio, which involves only the likelihood, and the proposal $\boldsymbol{\xi}' = \sqrt{1-\beta^2}\,\boldsymbol{\xi} + \beta\boldsymbol{\zeta}$ preserves it exactly — so the acceptance rate does not degenerate with grid refinement. Four test observations, two chains each, 6000 burn-in iterations with the step size adapted to a 25% acceptance target, then 20,000 sampling iterations. One forward solve per chain per iteration.

It did not converge, and that is the measurement. The sampler behaved as designed — acceptance settled at 0.20 with $\beta = 0.044$ — but 52,000 forward solves per observation were not enough. Split-$\hat{R}$ on the potential runs from 1.05 to 1.89, and the effective sample size from 20,000 draws is between 12 and 39. The blunt diagnostic is the one on the quantity the comparison would have used: two chains targeting the *same* posterior disagree on the posterior mean by 0.38 to 0.99 relative L^2, against 1.04 for two chains targeting *different* observations. On the worst observation, running the same problem twice is almost as different as running two unrelated problems. No posterior comparison is drawn from this, and none of the agreement numbers the run recorded are quoted, because a reference that disagrees with itself is not a reference.

What the failure does establish. Both methods were timed on the same machine and the same backend. The chain cost 701 seconds per observation and did not produce a usable posterior; the trained flow produces its $K = 32$ ensemble in 16.1 seconds per observation, a factor of 43, and needs *zero* forward solves to do it. The hardware-independent form of that statement is the useful one: sampling the posterior classically cost 52,000 PDE solves per observation here and bought a chain that had not mixed, while the amortized sampler pays its solves once, at dataset-construction time, and then answers any number of observations. This is the offline/online split of Section 2.2 as a measurement rather than an argument, and it is the strongest form in which this report can make it.

What it does not establish. That vanilla pCN at this budget fails is not that Markov chain Monte Carlo is infeasible on this problem. pCN takes the same step size in all 4096 directions, and the observation constrains only a few hundred of them; a sampler exploiting that structure — dimension reduction onto the leading prior modes, or a gradient-informed proposal using the adjoint of the solver — would very likely mix at a budget this one did not. The honest position is that the accuracy of the learned posterior remains unverified, that verifying it is expensive, and that the expense is itself the argument for the amortized method. Section 5 treats this as the report's main open question rather than a settled comparison.

4.10 Threats to validity

Replication is partial. E2, E4 and E7 were rerun at three matched seeds, and in both places where a seed variance could be measured it was small against the effect (0.5–1.8% on the Burgers backbones, 1.6–5.0% on the components) — with the striking exception of maximum-likelihood training at 39.3%. E3, E5, E6, E8 and E9 remain single runs at a seed that differs between variants, so strictly Section 4.3's Darcy panel compares ConvNet-at-seed-42 with UNet-at-seed-2042. The Burgers replication makes that a mild concern rather than an open one, but it does not close it, and the Darcy replication was started and abandoned.

No deterministic baseline. Nothing here is compared against a regressor trained on the same data; the Fourier neural operator reaches roughly one per cent on the standard benchmarks [19], though the Darcy variant used here is the harder parametric one and is not directly comparable to that figure.

The learned posterior is unverified. E10 measured what a classical reference costs but did not obtain one (Section 4.9), so no result here shows that the inverse samplers agree with the Bayesian posterior — only that they are calibrated against the truth and use their observation. This is the largest single gap in the report, and closing it needs a better sampler rather than more of the same one.

One dataset size on the forward problems. Every forward experiment uses 3000 samples, and Section 4.7 shows the training budget alone can decide a conclusion. The inverse side now varies observation quality (E9) and, across experiments, budget and data size, but no forward experiment varies either.

Backbones matched by configuration, not budget. They differ by an order of magnitude in parameters and a factor of two and a half in cost per evaluation, in opposite directions: the ConvNet runs at full resolution while the UNet downsamples, so on Darcy the ConvNet is both more accurate *and* more

expensive, at 0.43 s per training step against 0.17 s. "Smaller is better here" is about parameters, not compute. Matched-parameter or matched-FLOP comparisons are different experiments and might order them differently.

The objective comparison is one budget on one small problem. Whether more probes or more epochs would close the sample-quality gap is untested; the variants that would have answered it did not complete.

The component ablation is one problem. E7 ran on Burgers 1D because the whole matrix is affordable there. Whether logit-normal sampling buys the same few-step accuracy on Darcy, where the trajectories are far more curved, is not tested.

Measurement details. The original runs averaged metrics per batch, the re-evaluations per sample; the two agree within a few per cent (E3's ConvNet is 0.0734 recorded, 0.0701 recomputed). The R3 and E9 uncertainty suites use different numbers of test conditions and different ensemble seeds, so Table 4 marks which protocol produced each row. Wall-clock numbers come from two machines and are comparable only within a table.

Reproducibility of E5. Its runner was not committed alongside the run and was later reconstructed from the resolved configurations, so the configuration is reproducible but the weights in hand were produced by code that no longer exists in exactly that form. One cell — κ-only null at 5000 — saved no checkpoint, which is why the κ null baseline in Table 4 is taken from E9's reference cell instead. Against that, R3 re-derived the point metrics for every cell that has weights and reproduced the original win rates to within a few per cent, which is why those numbers are used at all.

5 Conclusions

This report set out to do two things: describe a library in which the axes of the design space are orthogonal components rather than forks of a codebase, and use it to measure some of them. It is worth separating what the measurements support from what the architecture claims.

What was measured. Replacing the conditioner with one that discards its input costs a factor of 17 in median relative L^2 while leaving the loss curve looking healthy, which is the concrete case for selecting on sampled error (Section 4.2). On both forward problems the smallest of four backbones is the most accurate — by 12.4× on Burgers — and attention changes nothing beyond seed noise, so capacity is not the binding constraint at this scale (Section 4.3). Fitting the same flow by exact maximum likelihood rather than flow matching improves test likelihood from -1.21 to -1.86 nats per dimension while degrading sample accuracy from 0.61 to 1.06 relative L^2, at 70× the wall-clock and 8.9× the memory: each objective wins on the metric it optimizes, and the flow/objective separation is what makes that a controlled comparison rather than a comparison between two codebases (Section 4.6). Replicated at three seeds, that comparison gains a second axis — the simulation-based objective varies by 39% across seeds against the simulation-free one's 4%, so it is not only slower but less reproducible at this budget.

The sampler is an axis, and does not behave classically. Sweeping solver and step count on frozen weights gave the three least expected results (Section 4.4). First-order Euler beats fourth-order Runge–Kutta at matched cost on every model and both problems, by up to 8.6×, because a high-order tableau earns its order by probing the field away from the trajectory and a learned field is not accurate there. The *slope* of the error-against-cost curve separates discretization error from model error, making the sweep a diagnostic rather than a price list. And integrating the learned ODE to convergence is not the objective: on two of eight models the best fixed-step error is 6–11% below the converged one, because truncation error partially cancels the model's. Quoting an accuracy "at 50 steps" therefore quotes one point on a non-monotone, model-dependent curve. Timed fairly against the solver it is meant to replace, the surrogate is 1.9× slower at its cheapest setting and 65× at its most accurate; on a linear elliptic problem at this size it does not pay for itself, and the case for the method rests on the inverse direction instead.

On the inverse problem, the metric decided the conclusion. In both matched comparisons of a conditioned model against its unconditional counterpart, the proper scoring rules register roughly twice the effect that a single draw's relative L^2 does — 18.6% against 12.8% on the coefficient target, 25.4% against 13.8% on the joint one. A metric comparing one sample against one truth cannot distinguish a model that learned a broad posterior from one that learned nothing, and half of what conditioning bought here is invisible to it. The same suite caught the report's sharpest negative result: training the coefficient model five times longer improved its single-draw error by 15% while moving its spread–skill ratio from 0.92 to 0.47 and its 90% coverage from 0.84 to 0.70. Additional optimization bought sharpness the evidence does not support — a failure a point metric rewards and a proper scoring rule penalizes, and one this report previously misattributed to training-set size before the budget and the data were separated.

Forward spread is model error, and that is measurable. Two forward models approximating the same degenerate posterior report spread–skill ratios of 1.16 and 0.22. Since the true posterior is a point mass and identical for both, the factor of five is a property of the models, which converts Remark 5 from an argument into a measurement. Averaging 32 draws halves the forward error; and forward spread predicts the model's own error well ($\rho \approx 0.8$) while inverse spread does not. Sweeping the observation quality shows this is a continuum rather than a dichotomy: the rank correlation between spread and error runs from -0.54 at 5% of the grid observed to $+0.54$ at full observation, changing sign as the problem becomes determined. Sampler spread is a usable error estimate exactly where it is not a posterior width — that is, exactly where it is least needed. Finally, the learned inverse posterior carries the spatial structure the physics implies: binned by local flux magnitude its width falls by 3.9× from the flattest to the steepest decile, without ever being told that κ is identifiable only where ∇u is large.

What the architecture is worth. Every comparison above was a configuration change. Swapping the conditioner, the backbone or the objective touched one argument; sweeping the solver and step count touched none at all, because those are call-time arguments on frozen weights; and scoring 32-member ensembles reused a module the training code never imports. That is the argument for the separation of concerns, and it is what the experiments genuinely support: not that the library is fast or accurate, but that the marginal cost of asking one more question of the design space is small.

The counterweight is that the same experiments exposed defects the design did not prevent. The finite-ensemble correction in the spread–skill ratio was applied in the wrong direction and survived its unit test, because that test used $K = 50$ where the correct and incorrect forms differ by two per cent; it is now pinned at $K = 2, 4, 8, 32$, where they differ by twenty-two. The inverse experiment's runner was not committed alongside its run and had to be reconstructed afterwards from the resolved configurations. And the named-configuration module intended to make benchmarks citable does not import. None of these is visible from the architecture diagram — orthogonal components make a design space navigable, but do not make its individual measurements correct.

What remains. The two axes the library was built to make comparable have now been compared, and both returned less than the literature's framing promised. Varying the path, the time schedule and the coupling moves few-step accuracy by 22% and trajectory straightness by a factor of two, but the gain is a *trade* against converged accuracy rather than an improvement, and the optimal-transport coupling — the variant with the strongest theoretical case — was worse at every sampling budget, having straightened the training interpolant while leaving the learned trajectories curved. Reflow straightened as expected and bought no accuracy, and its deliberately decoupled control straightened too, so straightening is not by itself evidence that the pairing invariant was honoured. Reporting these is the point: the components are configurations, which is what made six of them affordable to try, and a design that makes negative results cheap to obtain is doing its job.

The comparison the inverse problem was posed against was attempted and did not finish. A preconditioned Crank–Nicolson chain targets exactly the right posterior here, because the prior is Gaussian in the generator's own white-noise coordinates; at 52,000 forward solves per observation it had still not mixed, with two chains on one observation disagreeing about the posterior mean almost as much as two chains on different observations (Section 4.9). So the report cannot say that the learned posterior is the Bayesian one — only that it is calibrated and uses its data. What the failure does buy is the amortization argument in measured form: on the same machine, the chain cost 701 seconds per observation and produced nothing usable, while the trained flow produced its 32-member ensemble in 16 and needed no forward solves at all. That a reference posterior is expensive enough to be out of reach is the strongest available statement of why one would want an amortized sampler, and it is also the report's largest open question: verifying the posterior needs a sampler that exploits the low-dimensional structure of the likelihood, not more iterations of this one. Beyond it, the remaining gaps listed in Section 4.10 — no deterministic baseline, one training-set size on the forward problems, and replication on five of the nine experiments — need no new idea, only compute and the discipline of holding the seed fixed while varying the thing under study.

References

[1] M. S. Albergo and E. Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations (ICLR)*, 2023.

[2] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: Composable transformations of Python+NumPy programs, 2018. `http://github.com/jax-ml/jax`.

[3] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[4] H. Chung, J. Kim, M. T. Mccann, M. L. Klasky, and J. C. Ye. Diffusion posterior sampling for general noisy inverse problems. In *International Conference on Learning Representations (ICLR)*, 2023.

[5] S. L. Cotter, G. O. Roberts, A. M. Stuart, and D. White. MCMC methods for functions: Modifying old algorithms to make them faster. *Statistical Science*, 28(3):424–446, 2013. doi: 10.1214/13-STS421.

[6] J. R. Dormand and P. J. Prince. A family of embedded Runge–Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.

[7] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, D. Podell, T. Dockhorn, Z. English, K. Lacey, A. Goodwin, Y. Marek, and R. Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning (ICML)*, 2024.

[8] V. Fortin, M. Abaza, F. Anctil, and R. Turcotte. Why should ensemble spread match the RMSE of the ensemble mean? *Journal of Hydrometeorology*, 15(4):1708–1713, 2014.

[9] S. Fresca and A. Manzoni. POD-DL-ROM: Enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition. *Computer Methods in Applied Mechanics and Engineering*, 388:114181, 2022.

[10] T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

[11] W. Grathwohl, R. T. Q. Chen, J. Bettencourt, I. Sutskever, and D. Duvenaud. FFJORD: Free-form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations (ICLR)*, 2019.

[12] T. M. Hamill. Interpretation of rank histograms for verifying ensemble forecasts. *Monthly Weather Review*, 129(3): 550–560, 2001.

[13] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[14] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[15] M. F. Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics — Simulation and Computation*, 18 (3):1059–1076, 1989.

[16] F. Koehler, S. Niedermayr, R. Westermann, and N. Thuerey. APEBench: A benchmark for autoregressive neural emulators of PDEs. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[17] G. Kohl, K. Um, and N. Thuerey. Benchmarking autoregressive conditional diffusion models for turbulent flow simulation. In *International Conference on Machine Learning (ICML)*, 2024.

[18] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. *Journal of Machine Learning Research*, 24(89): 1–97, 2023.

[19] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[20] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

[21] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu, and I. Gat. Flow matching guide and code. *arXiv preprint arXiv:2412.06264*, 2024.

[22] P. Lippe, B. S. Veeling, P. Perdikaris, R. E. Turner, and J. Brandstetter. PDE-Refiner: Achieving accurate long rollouts with neural PDE solvers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[23] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.

[24] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[25] I. Price, A. Sanchez-Gonzalez, F. Alet, T. R. Andersson, A. El-Kadi, D. Masters, T. Ewalds, J. Stott, S. Mohamed, P. Battaglia, R. Lam, and M. Willson. Probabilistic weather forecasting with machine learning. *Nature*, 637:84–90, 2025.

[26] A. Quarteroni, A. Manzoni, and F. Negri. *Reduced Basis Methods for Partial Differential Equations: An Introduction*, volume 92 of *UNITEXT*. Springer, 2016.

[27] D. J. Rezende and S. Mohamed. Variational inference with normalizing flows. In *International Conference on Machine Learning (ICML)*, pages 1530–1538, 2015.

[28] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.

[29] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[30] A. M. Stuart. Inverse problems: A Bayesian perspective. *Acta Numerica*, 19:451–559, 2010.

[31] G. J. Székely and M. L. Rizzo. Energy statistics: A class of statistics based on distances. *Journal of Statistical Planning and Inference*, 143(8):1249–1272, 2013.

[32] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *Transactions on Machine Learning Research*, 2024.

[33] J. Wildberger, M. Dax, S. Buchholz, S. R. Green, J. H. Macke, and B. Schölkopf. Flow matching for scalable simulation-based inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.